

IDP Performance Testing – Dynamic Test Account Setup & MFA Disable Playbook



Owner: QA Automation (IDP/UE)
Environments: Stage1 (primary), QC (optional)
DBs: AGSAuthStage1, AGSAuthQC4/QC1
Servers: [GWQMSSQL02.rsd.crumprsd.com](https://gwqmssql02.rsd.crumprsd.com) (Stage1), [GWQMSSQL01.rsd.crumprsd.com](https://gwqmssql01.rsd.crumprsd.com) (QC)
Auth: Windows Authentication (SSMS), "Trust server certificate" enabled

What this page covers

1. **Connect** to SQL Server used by IDP
2. **Pick a plan dynamically** and locate its identities
3. **Recycle / migrate** an existing test identity (no need to create a new one)
4. **Disable MFA** safely for that identity (and later restore)
5. **Optionally assign multiple plans** to the same user for coverage
6. **Set known password** and ensure phone on file
7. **End-to-end verification** & rollback

This procedure is derived from your team's working notes & commands (now formalized here).

Prerequisites

- SSMS installed and connection verified (see "SQL Server Access & SSMS Setup").
- SQL access: **Database Reader + Writer** on Stage1/QC.
- Approved **test users only**. Never apply to production.
- Postman collection for "Account Conversion/Migration" (see Step 6).

Variables you can change per run

Paste once into a new SSMS query window and adjust values.

```
/* ==== RUN VARIABLES (EDIT THESE) ==== */
DECLARE @EnvName      varchar(16)  = 'Stage1';      -- 'Stage1' | 'QC'
DECLARE @PlanCode     varchar(64)  = 'northdakota'; -- any valid plan code
DECLARE @UserToRecycle varchar(256) = 'pramodtest100'; -- existing test login to recycle
DECLARE @RunTag       varchar(64)  = '2025-Perf-A'; -- for auditing

/* Baseline account whose password hash we will copy (Newton@123 in notes) */
DECLARE @BaselineUser varchar(256) = 'TEST_84G19ULD09BG1NNDKR10';

/* New/migrated target username that should be usable after the flow */
DECLARE @TargetUser   varchar(256) = 'TEST_MNVJHOMEMPAHQAWVAL4F';
```

1) Connect to the right DB

Open **SSMS** connect to:

- Stage1: [GWQMSSQL02.rsd.crumprsd.com](https://gwqmssql02.rsd.crumprsd.com)
- QC: [GWQMSSQL01.rsd.crumprsd.com](https://gwqmssql01.rsd.crumprsd.com)

Select the database (e.g., **AGSAuthStage1**) in the dropdown.

2) Find the plan's source record & its identities (dynamic)

```

/* Locate the plan source row */
SELECT *
FROM dbo.ExternalSource
WHERE SourceCode = @PlanCode;

/* Usually you'll need the ExternalId/Supplier GUID from that row.
   Example pattern to pull credentials tied to that ExternalId: */
DECLARE @ExternalId uniqueidentifier =
(
    SELECT TOP (1) CAST(ExternalId AS uniqueidentifier)
    FROM dbo.ExternalSource
    WHERE SourceCode = @PlanCode
);

SELECT
    pc.ID                AS CredentialId,
    pc.ProfileId,
    pc.UserName,
    pp.RowGuid
FROM person.Credential pc
JOIN person.Profile    pp ON pp.ProfileId = pc.ProfileId
WHERE pc.ExternalId = @ExternalId;

```

*Use this to pick **which user(s)** you'll recycle or migrate under the chosen plan.*

3) Create a lightweight audit trail (one-time, then reuse)

```

IF NOT EXISTS (SELECT 1 FROM sys.schemas WHERE name = 'qa_audit')
    EXEC('CREATE SCHEMA qa_audit AUTHORIZATION dbo;');

IF NOT EXISTS (SELECT 1 FROM sys.tables WHERE name = 'IDP_Run_Audit' AND schema_id = SCHEMA_ID('qa_audit'))
BEGIN
    CREATE TABLE qa_audit.IDP_Run_Audit
    (
        AuditId        bigint IDENTITY PRIMARY KEY,
        RunTag         varchar(64) NOT NULL,
        EnvName        varchar(16) NOT NULL,
        UserName       varchar(256) NOT NULL,
        ProfileId      bigint NULL,
        WhenAt         datetime2 NOT NULL DEFAULT SYSUTCDATETIME(),
        WhoBy          varchar(128) NOT NULL DEFAULT SYSTEM_USER,
        Note           varchar(4000) NULL
    );
END;

```

4) Recycle the chosen identity (safe delete of old ties)

Removes org/profile artifacts so we can re-hydrate the account cleanly for this plan.

```

SET XACT_ABORT ON;
BEGIN TRAN;

-- 4.1 ProfileOrganization ties (clean out org mappings)
DELETE po
FROM Person.ProfileOrganization po
JOIN person.Credential pc ON pc.ProfileId = po.ProfileId
WHERE pc.UserName = @UserToRecycle;

-- 4.2 Credential audit history
DELETE ca
FROM Person.CredentialAudit ca
JOIN person.Credential pc ON pc.ID = ca.CredentialId
WHERE pc.UserName = @UserToRecycle;

-- 4.3 Credential row itself
DELETE FROM person.Credential
WHERE UserName = @UserToRecycle;

-- 4.4 Profile row (only if *no* other creds depend on it)
-- Replace 1122200 if you already know a specific ProfileId; otherwise lookup first:
-- SELECT ProfileId FROM person.Profile WHERE <criteria>;
-- Example:
-- DELETE FROM person.Profile WHERE ProfileId = 1122200;

COMMIT TRAN;

-- Log
INSERT INTO qa_audit.IDP_Run_Audit (RunTag, EnvName, UserName, Note)
VALUES (@RunTag, @EnvName, @UserToRecycle, 'Recycled identity by removing PO/Credential/Audit ties');

```

These exact commands match your team's working approach. Adjust the profile delete if that profile is shared.

5) (Optional) Prepare multi-plan coverage for one user

If your schema links users to plans via a mapping table (e.g., UserPlan or Person.ProfileOrganization), insert rows for the plan(s) you need.

```

/* Example: using a mapping list you edit each run */
DECLARE @PlanMap TABLE (UserName varchar(256), PlanCode varchar(64));
INSERT INTO @PlanMap VALUES
(@TargetUser, @PlanCode), -- current plan
(@TargetUser, 'anotherplan'); -- add more plans if needed

/* INSERT into the right mapping table for your schema.
   If you use ProfileOrganization, you'll need ProfileId & OrgId for those plans. */
-- Pseudocode (replace with your actual columns):
/*
INSERT INTO Person.ProfileOrganization (ProfileId, OrganizationId, IsPrimary, CreatedAt, CreatedBy)
SELECT p.ProfileId, es.OrganizationId, CASE WHEN ROW_NUMBER() OVER (PARTITION BY p.ProfileId ORDER BY es.
OrganizationId)=1 THEN 1 ELSE 0 END,
SYSUTCDATETIME(), SYSTEM_USER
FROM @PlanMap pm
JOIN person.Credential c ON c.UserName = pm.UserName
JOIN person.Profile p ON p.ProfileId = c.ProfileId
JOIN dbo.ExternalSource es ON es.SourceCode = pm.PlanCode
LEFT JOIN Person.ProfileOrganization po ON po.ProfileId = p.ProfileId AND po.OrganizationId = es.OrganizationId
WHERE po.ProfileId IS NULL;
*/

```

*Keep this section as a **template** and point it to your real mapping table.*

6) Account conversion / migration (Postman)

Use the **IDP Account Migration** API/collection your team already has (Postman). Trigger the migration for the recycled identity under the chosen plan.

*Specific endpoint/params are in your existing collection; keep the run **tagged** with @RunTag in the request description for traceability.*

7) Set password on the migrated account (copy hash from baseline)

```
/* Copy the password/hash from a known baseline account to the target account */
UPDATE person.Credential
SET Password =
(
    SELECT Password
    FROM person.Credential
    WHERE UserName = @BaselineUser
)
WHERE UserName = @TargetUser;

/* Note from team doc: the baseline account above corresponds to password: Newton@123 */
```

This exactly mirrors your existing working step.

8) Ensure phone # is present for the account

Your doc requires that the migrated account has a **phone number** on file.

Add/verify it using your environment's contact table(s) (keep these placeholders until you map to the real tables):

```
/* Example pattern: replace with your real contact tables/columns */
-- UPDATE person.Profile SET PhoneNumber = '7175551212' WHERE ProfileId = (SELECT ProfileId FROM person.
Credential WHERE UserName = @TargetUser);
```

9) Disable MFA cleanly for test runs (and restore later)

*We want to temporarily **turn off MFA** for selected test accounts so that performance/regression runs can proceed without prompts. Multiple options are available—choose whichever applies best to your schema.*

9A) Policy toggle (if UserPolicy table is used)

```
UPDATE pol SET pol.MfaRequired = 0, pol.UpdatedAt = SYSUTCDATETIME(), pol.UpdatedBy = SYSTEM_USER FROM security.
UserPolicy pol JOIN person.Credential c ON c.UserId = pol.UserId WHERE c.UserName IN ('GPKUsername5824');
```

9B) Factor deactivation + session cleanup

```
-- Deactivate MFA factors UPDATE f SET f.IsActive = 0, f.Status = 'Disabled', f.RevokedAt = SYSUTCDATETIME(), f.
RevokedBy = SYSTEM_USER FROM security.MfaFactor f JOIN person.Credential c ON c.UserId = f.UserId WHERE c.
UserName IN ('GPKUsername5824') AND f.IsActive = 1; -- Remove in-flight sessions DELETE s FROM security.
UserSession s JOIN person.Credential c ON c.UserId = s.UserId WHERE c.UserName IN ('GPKUsername5824');
```

9C) Direct profile flag toggle (simple + reliable)

```
UPDATE person.Profile SET TwoFactorEnabled = 0 WHERE ProfileId IN ( SELECT ProfileId FROM person.Credential WHERE UserName IN ('GPKUsername5824') );
```

9D) Ensure a phone number is present

Some flows require a phone # on the profile even if MFA is disabled. Update it if missing:

```
UPDATE person.Profile SET PhoneNumber = '+19782897549' WHERE ProfileId IN (11001);
```

9E) Quick validation

```
SELECT c.UserName, p.ProfileId, p.TwoFactorEnabled, p.PhoneNumber FROM person.Credential c JOIN person.Profile p ON p.ProfileId = c.ProfileId WHERE c.UserName IN ('GPKUsername5824');-- You should see TwoFactorEnabled = 0 and a valid phone number.
```

10) Login verification

Attempt an IDP login with:

- **Username:** @TargetUser
- **Password:** same as the baseline (per team note: Newton@123)
- Confirm: **no MFA prompt** appears and flows execute for your selected **plan(s)**.

11) Restore MFA (post-run)

```
UPDATE pol
SET pol.MfaRequired = 1, pol.UpdatedAt = SYSUTCDATETIME(), pol.UpdatedBy = SYSTEM_USER
FROM security.UserPolicy pol
JOIN person.Credential c ON c.UserId = pol.UserId
WHERE c.UserName = @TargetUser;

UPDATE f
SET f.IsActive = 1, f.Status = 'Active', f.ReactivatedAt = SYSUTCDATETIME(), f.ReactivatedBy = SYSTEM_USER
FROM security.MfaFactor f
JOIN person.Credential c ON c.UserId = f.UserId
WHERE c.UserName = @TargetUser AND f.RevokedAt IS NOT NULL;

/* Log the closure of the run */
INSERT INTO qa_audit.IDP_Run_Audit (RunTag, EnvName, UserName, Note)
VALUES (@RunTag, @EnvName, @TargetUser, 'MFA restored; run completed');
```

Troubleshooting

- **Still seeing MFA prompt** clear user sessions; verify MfaRequired = 0; check app-level overrides.
- **Login fails** ensure credential row exists for @TargetUser; confirm password copy succeeded.
- **Wrong plan data** confirm you pulled the correct ExternalId for @PlanCode and that any plan-to-user mapping rows exist.

Appendix – “Recycle & Migrate” quick script (one user, one plan)

*Use when you just need a **single** user set up quickly for a different plan.*

```
/* 1) Lookup plan */
SELECT * FROM dbo.ExternalSource WHERE SourceCode = @PlanCode;

/* 2) Recycle old ties for @UserToRecycle (PO, Audit, Credential), then run migration via Postman */

/* 3) Set password on @TargetUser from @BaselineUser */
UPDATE person.Credential
SET Password = (SELECT Password FROM person.Credential WHERE UserName = @BaselineUser)
WHERE UserName = @TargetUser;
```

Exact statements above originate from your internal notes and have been consolidated here for repeatability.

Final notes

- Keep @PlanCode flexible—this page is designed to be **plan-agnostic**.
- If your MFA schema uses different table/column names, update the **Section 9** snippets once, then **lock them in** as your team's standard.
- Always log each run with @RunTag for easy audit & rollback.